



AWS Session

Summary 18-05-2023

- Amazon **Elastic Kubernetes Service (EKS)** is a managed Kubernetes service that makes it easy to run Kubernetes on AWS. It is managed service, so you don't need to worry about installing, configuring, or managing Kubernetes.
- In Amazon EKS, AWS manages the **control plane or master node** of the EKS cluster for you. AWS takes care of provisioning, managing, and maintaining the control plane for you. This includes managing the underlying infrastructure, ensuring high availability of the control plane components, and performing updates and patches.
- And you are responsible for managing the **worker nodes** that run your containers. You need to provision and manage EC2 instances or use EC2 Auto Scaling Groups to ensure that the desired number of worker nodes are available to run your containers.
- If you want to manage both the control plane and the worker nodes, **AWS Fargate** is the service that comes into the picture.
- **AWS Fargate** is a serverless compute engine for containers provided by AWS. It allows you to run containers without managing the underlying infrastructure. With Fargate, you don't need to provision or manage EC2 instances for your worker nodes.
- When using Amazon **EKS with Fargate**, AWS manages both the control plane and the worker nodes for you. You no longer have to worry about provisioning, scaling, or patching the worker nodes. Fargate automatically takes care of the infrastructure management, including scaling the compute resources based on your container requirements.

[AWS]

- Create an EKS cluster with Fargate

```
C:\Users\Vimal Daga>eksctl create cluster --name lwcluster1 --fargate
```

- View your cluster nodes

```
C:\Users\Vimal Daga>kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
fargate-ip-192-168-144-60.ap-south-1.compute.internal	Ready	<none>	6m17s	v1.24.12-eks-f4dc2c0
fargate-ip-192-168-177-30.ap-south-1.compute.internal	Ready	<none>	6m16s	v1.24.12-eks-f4dc2c0

- When you create an EKS cluster with Fargate then, by default, two nodes are launched for internal purposes. These nodes are used to run the Kubernetes control plane, which is responsible for managing the cluster's resources.
- View namespaces and pods within that namespace

```
C:\Users\Vimal Daga>kubectl get ns
```

NAME	STATUS	AGE
default	Active	22m
kube-node-lease	Active	22m
kube-public	Active	22m
kube-system	Active	22m

```
C:\Users\Vimal Daga>kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-79dbbf8786-sggpd	1/1	Running	0	10m
coredns-79dbbf8786-zqqr1	1/1	Running	0	10m

- View pods with the node name

```
C:\Users\Vimal Daga>kubectl get pods -o wide -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
coredns-79dbbf8786-sggpd	1/1	Running	0	11m	192.168.144.60	fargate-ip-192-168-144-60.ap-sout
h-1.compute.internal	<none>	<none>	<none>			
coredns-79dbbf8786-zqqr1	1/1	Running	0	11m	192.168.177.30	fargate-ip-192-168-177-30.ap-sout
h-1.compute.internal	<none>	<none>	<none>			

- Now create a deployment

```
C:\Users\Vimal Daga>kubectl create deployment myd --image=vimal13/apache-webserver-php
deployment.apps/myd created
```

[AWS]

- When you create a pod in an Amazon EKS cluster with Fargate, the pod will initially be in the Pending state. This is because the Fargate scheduler needs to first launch a Worker node before it can launch the pod. Once the Worker node is launched, the pod will transition to the Running state.

```
C:\Users\Vimal Daga>kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
myd-56d496f844-8bvvh 0/1     Pending   0           4s
```

- We can see that a new node and pod have just been launched.

```
C:\Users\Vimal Daga>kubectl get nodes
NAME                                                    STATUS    ROLES    AGE   VERSION
fargate-ip-192-168-144-190.ap-south-1.compute.internal Ready    <none>   19s   v1.24.12-eks-f4dc2c0
fargate-ip-192-168-144-60.ap-south-1.compute.internal  Ready    <none>   13m   v1.24.12-eks-f4dc2c0
fargate-ip-192-168-177-30.ap-south-1.compute.internal  Ready    <none>   13m   v1.24.12-eks-f4dc2c0
```

```
C:\Users\Vimal Daga>kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
myd-56d496f844-8bvvh 1/1     Running   0           3m15s
```

- Now, if we try to scale the pods, we can observe that the node will also increase.

```
C:\Users\Vimal Daga>kubectl scale deployment myd --replicas=2
deployment.apps/myd scaled
```

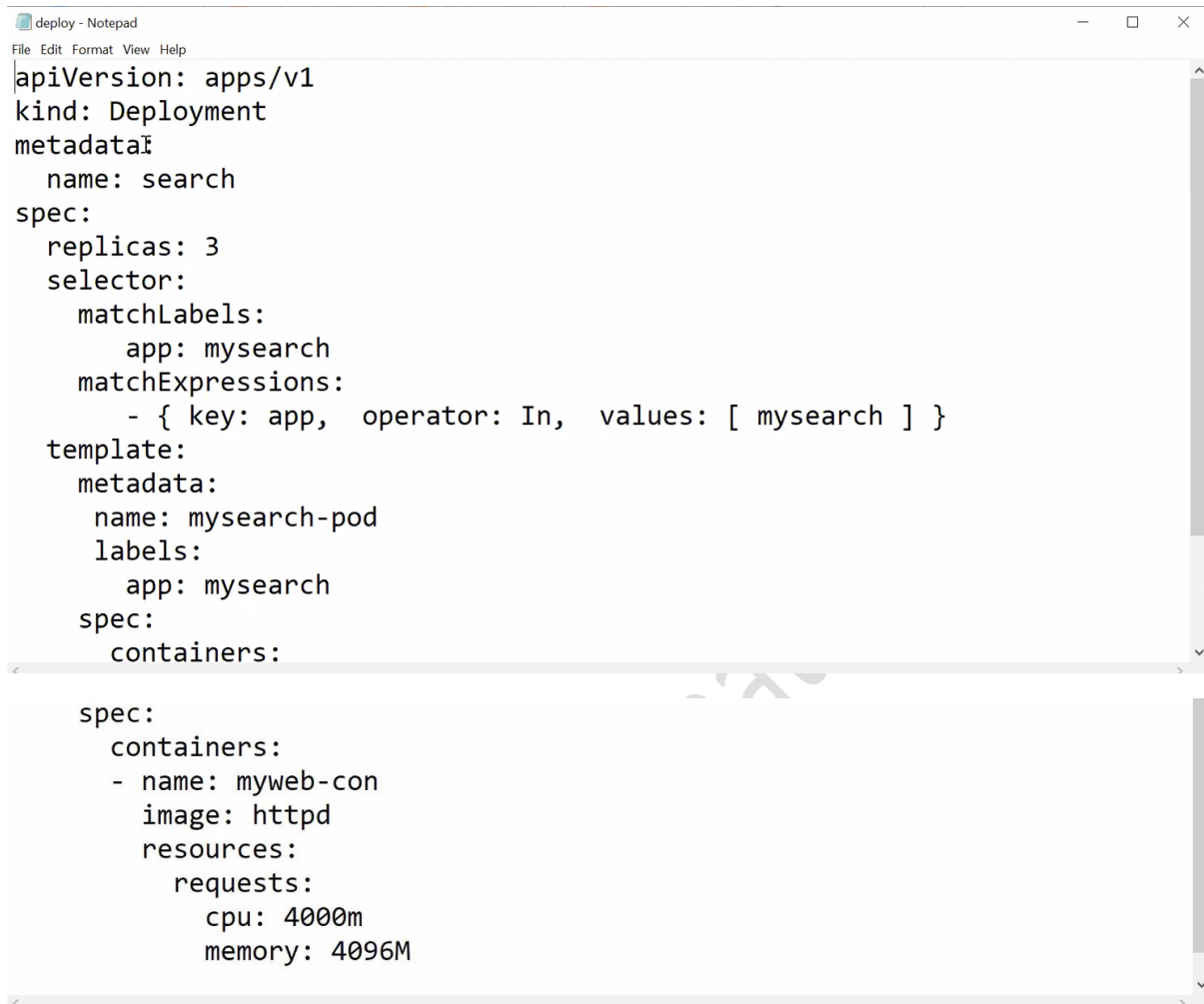
```
C:\Users\Vimal Daga>kubectl get nodes
NAME                                                    STATUS    ROLES    AGE   VERSION
fargate-ip-192-168-107-222.ap-south-1.compute.internal Ready    <none>   5s    v1.24.12-eks-f4dc2c0
fargate-ip-192-168-144-190.ap-south-1.compute.internal Ready    <none>   7m32s v1.24.12-eks-f4dc2c0
fargate-ip-192-168-144-60.ap-south-1.compute.internal  Ready    <none>   20m   v1.24.12-eks-f4dc2c0
fargate-ip-192-168-177-30.ap-south-1.compute.internal  Ready    <none>   20m   v1.24.12-eks-f4dc2c0
```

```
C:\Users\Vimal Daga>kubectl get pods
NAME                READY   STATUS             RESTARTS   AGE
myd-56d496f844-7cn6g 0/1     ContainerCreating   0           48s
myd-56d496f844-8bvvh 1/1     Running             0           8m11s
```

```
C:\Users\Vimal Daga>kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
myd-56d496f844-7cn6g 1/1     Running   0           73s
myd-56d496f844-8bvvh 1/1     Running   0           8m36s
```

[AWS]

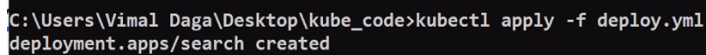
- We are going to put some requests for resources in the document below.



```
deploy - Notepad
File Edit Format View Help
apiVersion: apps/v1
kind: Deployment
metadata:
  name: search
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mysearch
    matchExpressions:
      - { key: app, operator: In, values: [ mysearch ] }
  template:
    metadata:
      name: mysearch-pod
      labels:
        app: mysearch
    spec:
      containers:

spec:
  containers:
    - name: myweb-con
      image: httpd
      resources:
        requests:
          cpu: 4000m
          memory: 4096M
```

- Deploy the application.



```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl apply -f deploy.yml
deployment.apps/search created
```

- We will get a new node and pod as soon as we deploy.

[AWS]

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl get nodes
NAME                                                    STATUS    ROLES    AGE   VERSION
fargate-ip-192-168-107-222.ap-south-1.compute.internal Ready    <none>   5m59s v1.24.12-eks-f4dc2c0
fargate-ip-192-168-123-84.ap-south-1.compute.internal Ready    <none>   35s    v1.24.12-eks-f4dc2c0
fargate-ip-192-168-144-190.ap-south-1.compute.internal Ready    <none>   13m    v1.24.12-eks-f4dc2c0
fargate-ip-192-168-144-60.ap-south-1.compute.internal Ready    <none>   26m    v1.24.12-eks-f4dc2c0
fargate-ip-192-168-177-30.ap-south-1.compute.internal Ready    <none>   26m    v1.24.12-eks-f4dc2c0

C:\Users\Vimal Daga\Desktop\kube_code>kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
myd-56d496f844-7cn6g                1/1     Running   0           6m43s
myd-56d496f844-8bvvh                1/1     Running   0           14m
search-6c7dd4bd6-d892w              1/1     Running   0           79s
```

- And if we describe the pod, we can see requests for resources, and the node will also launch with some default resources in order to meet those requests.

kubectl describe pods search-6c7dd4bd6-d892w

```
IP: 192.168.123.84
IPs:
  IP: 192.168.123.84
Controlled By: ReplicaSet/search-6c7dd4bd6
Containers:
  myweb-con:
    Container ID:   containerd://cd1c1192330713a530418dcb0666d3288aa11553a4e92e2059a45be157a0073c
    Image:          httpd
    Image ID:       docker.io/library/httpd@sha256:90254ccc7352e1a5c8d1e4cdab2a032cef9fd5d4d632ca003a2943c9a9b0a3
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Thu, 18 May 2023 21:53:37 +0530
    Ready:          True
    Restart Count:  0
    Requests:
      cpu:          4
      memory:       4096M
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-xjbb5 (ro)
```

- Now, Create a namespace

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl create ns teama
namespace/teama created
```

- View your namespace

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl get ns
NAME                STATUS    AGE
default             Active    45m
kube-node-lease     Active    45m
kube-public          Active    45m
kube-system          Active    45m
teama                Active    3s
```

- Deploy the application in that created namespace.

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl create deployment myd1 --image=httpd -n teama
deployment.apps/myd1 created
```

- If we look at the description of the pod after the application has launched, we can see that the pod is failed.

```
kube-api-access-422z1:
  Type:               Projected (a volume that contains injected data from multiple sources)
  TokenExpirationSeconds: 3607
  ConfigMapName:       kube-root-ca.crt
  ConfigMapOptional:    <nil>
  DownwardAPI:         true
QoS Class:             BestEffort
Node-Selectors:         <none>
Tolerations:           node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                      node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason                  Age   From                  Message
  ----     -
Warning   FailedScheduling        32s   default-scheduler    0/5 nodes are available: 5 node(s) had untolerated taint {eks.amazonaws.com/compute-type: fargate}. preemption: 0/5 nodes are available: 5 Preemption is not helpful for scheduling.

C:\Users\Vimal Daga\Desktop\kube_code>kubectl get pods -n teama
NAME                                READY   STATUS    RESTARTS   AGE
myd1-5b86f498d7-8std5              0/1     Pending   0          47s
```

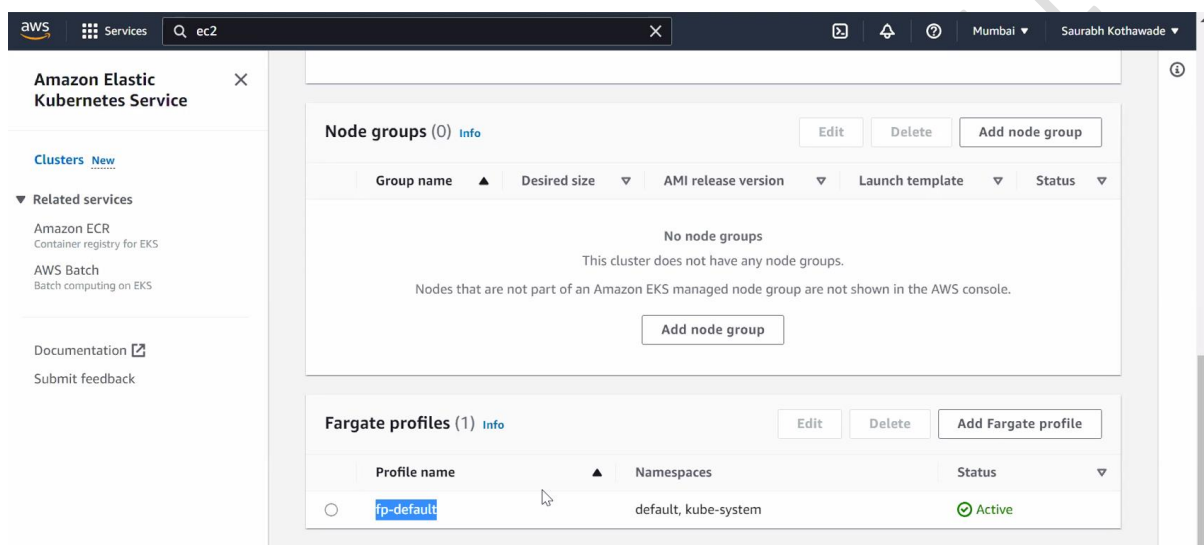
```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
fargate-ip-192-168-107-222.ap-south-1.compute.internal Ready    <none>   14m   v1.24.12-eks-f4dc2c0
fargate-ip-192-168-123-84.ap-south-1.compute.internal Ready    <none>   9m29s v1.24.12-eks-f4dc2c0
fargate-ip-192-168-144-190.ap-south-1.compute.internal Ready    <none>   22m   v1.24.12-eks-f4dc2c0
fargate-ip-192-168-144-60.ap-south-1.compute.internal Ready    <none>   35m   v1.24.12-eks-f4dc2c0
fargate-ip-192-168-177-30.ap-south-1.compute.internal Ready    <none>   35m   v1.24.12-eks-f4dc2c0
```

- It means that the namespace where the pod was scheduled has a taint that the pod is unable to tolerate. Taints are used to prevent pods from being scheduled on the specific namespace in a Kubernetes cluster, while tolerations allow pods to tolerate those taints and be scheduled on tainted nodes.
- Fargate in Amazon EKS only supports pods in the default namespace or kube-system namespace.
- **Fargate profile** in Amazon EKS is a configuration that defines which pods in your cluster should run on AWS Fargate. It allows you to specify

[AWS]

which namespaces and pod selectors should be associated with Fargate, enabling those pods to be launched and managed by the Fargate infrastructure.

- When you create an Amazon EKS cluster with Fargate, the default Fargate profile is automatically created for you. This default profile is configured to launch pods only in the default namespace and kube-system namespace. Pods in other namespaces will not be scheduled on Fargate by default.



- Create a Fargate profile for your namespace

```
C:\Users\Vimal Daga\Desktop\kube_code>eksctl create fargateprofile --name myfpteam --cluster lwcluster1 --namespace teama
2023-05-18 22:10:38 [i] creating Fargate profile "myfpteam" on EKS cluster "lwcluster1"
2023-05-18 22:11:12 [i] created Fargate profile "myfpteam" on EKS cluster "lwcluster1"
```

- If we launch an app inside that namespace after creating the fargate profile, it will launch successfully.

```
C:\Users\Vimal Daga\Desktop\kube_code>kubectl create deployment myd2 --image=httpd -n teama
deployment.apps/myd2 created
```

[AWS]

```
C:\Users\Vimal Daga\Desktop\kube_code>kubect1 get nodes
NAME
fargate-ip-192-168-107-222.ap-south-1.compute.internal
fargate-ip-192-168-112-187.ap-south-1.compute.internal
fargate-ip-192-168-123-84.ap-south-1.compute.internal
fargate-ip-192-168-144-190.ap-south-1.compute.internal
fargate-ip-192-168-144-60.ap-south-1.compute.internal
fargate-ip-192-168-177-30.ap-south-1.compute.internal
STATUS    ROLES    AGE    VERSION
Ready     <none>    25m    v1.24.12-eks-f4dc2c0
Ready     <none>    6s     v1.24.12-eks-f4dc2c0
Ready     <none>    20m    v1.24.12-eks-f4dc2c0
Ready     <none>    33m    v1.24.12-eks-f4dc2c0
Ready     <none>    46m    v1.24.12-eks-f4dc2c0
Ready     <none>    46m    v1.24.12-eks-f4dc2c0

C:\Users\Vimal Daga\Desktop\kube_code>kubect1 get pods -n teama
NAME                READY   STATUS    RESTARTS   AGE
myd1-5b86f498d7-8std5  0/1     Pending   0           12m
myd2-7669cf7c6d-ckx9d  1/1     Running   0           48s
```